PROJECT REPORT

Script-Controlled ACL – Restrict Record Access Based on Custom Script

ServiceNow Developer Instance

Team ID: SWTID-2026-5170
Date: 02 October 2026
Maximum Marks: 10

TABLE OF CONTENTS

1. INTRODUCTION 3

1.1 Project overview 3

1.2 Purpose 3

2. IDEATION PHASE 3

2.1 Problem statement 3

2.2 Empathy map canvas 3

2.3 Brainstorming 3

3. REQUIREMENT ANALYSIS 3

3.1 Customer journey map 3

3.2 Solution requirement 3

3.3 Data flow diagram 3

3.4 Technology stack 3

4. PROJECT DESIGN 3

4.1 Problem solution fit 3

4.2 Proposed solution 3

4.3 Solution architecture 4

5. PROJECT PLANNING AND SCHEDULING 4

5.1 Project planning 4

6. FUNCTIONAL AND PERFORMANCE TESTING 4

6.1 Performance testing 4

7. RESULTS 4

7.1 Output screenshots 4

8. ADVANTAGES AND DISADVANTAGES 4

8.1 Advantages 4

8.2 Disadvantages 5

9. CONCLUSION 5

9.1 Project summary 5

9.2 Key outcomes 5

10. FUTURE SCOPE 5

10.1 Enhancement opportunities 5

10.2 Future automation 5

11. APPENDIX 5

11.1 GitHub 5

11.2 Project demo link 5

1. INTRODUCTION

1.1 Project overview

This project restricts access to ServiceNow records with script-controlled ACLs. A custom script checks field values and user conditions, and the user can access the record only when the script evaluates to true.

1.2 Purpose

To give record-level control that role-only ACLs cannot provide, protect confidential data and let users work only with records they are allowed to.

2. IDEATION PHASE

2.1 Problem statement

Role-based ACLs cannot restrict access using record ownership, status or confidentiality, so users may see or change records that they should not.

2.2 Empathy map canvas

Users say: "Why can everyone see this record?" They think sensitive data must be protected and feel worried about leaks. Pains: broad access and hard audits. Gains: record-level control and easy testing.

2.3 Brainstorming

Six ideas were scored. Script-controlled ACLs scored highest (14 of 15) and were selected.

3. REQUIREMENT ANALYSIS

3.1 Customer journey map

Six steps: log in, open the list, view a record, create, edit and delete. Each step is checked by an ACL.

3.2 Solution requirement

Create users and roles; create the table; build Read, Create, Write and Delete ACL scripts; verify through impersonation. Scripts must return true or false without errors and avoid database queries.

3.3 Data flow diagram

User request -> platform -> ACL engine fetches ACL rules -> script returns true or false -> the record is returned or the request is denied.

3.4 Technology stack

Component	Technology
Platform	ServiceNow Personal Developer Instance
Logic	ACL scripts (server-side JavaScript)
Data	Custom table u_confidential_record; sys_user; sys_security_acl
Testing	Impersonate User

4. PROJECT DESIGN

4.1 Problem solution fit

Problem: uncontrolled access to records. Solution: script-controlled ACLs that combine role, ownership and field values.

4.2 Proposed solution

Four ACLs on the Confidential Record table (read, create, write, delete). Each script defaults to deny and grants access only when the defined conditions are met.

4.3 Solution architecture

User -> ServiceNow authentication -> ACL engine -> role and script evaluation -> access granted or denied.

5. PROJECT PLANNING AND SCHEDULING

5.1 Project planning

Sprint	Story points	Duration	Milestones covered
Sprint-1	18	6 days	1 Users and Roles, 2 Tables, 3 ACL-READ
Sprint-2	19	6 days	4 CREATE, 5 WRITE, 6 DELETE, 7 Conclusion

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance testing

13 test cases were defined: 10 functional and 3 performance. They cover each operation for admin, owner, standard and no-role users. Record the final pass counts in the testing sheets.

Testing type	Test cases	Passed	Failed
Functional	10	[ ]	[ ]
Performance	3	[ ]	[ ]

7. RESULTS

7.1 Output screenshots

Insert screenshot: Users and roles

Insert screenshot: ACL records and scripts

Insert screenshot: Impersonated user list view

8. ADVANTAGES AND DISADVANTAGES

8.1 Advantages

- Record-level control using field values and user conditions.
- Server-side, so it cannot be bypassed from the browser.
- Uses only built-in ServiceNow features.
- Covers Read, Create, Write and Delete.
- Easy to verify with impersonation.

8.2 Disadvantages

- Scripts must be maintained when the table design changes.
- Poorly written scripts can deny valid access or slow lists.
- Needs the security_admin role to create ACLs.
- Debugging ACLs takes care.

9. CONCLUSION

9.1 Project summary

Script-controlled ACLs successfully restrict record access based on custom logic for all four operations.

9.2 Key outcomes

- Users, roles and a custom table created.
- Four script-controlled ACLs implemented.
- Access matrix defined and tested through impersonation.

10. FUTURE SCOPE

10.1 Enhancement opportunities

- Field-level ACLs.
- Shared Script Include for access logic.
- Group-based rules.

10.2 Future automation

- Audit logging of denied access.
- Automatic notifications on access attempts.
- Dashboards for access reports.

11. APPENDIX

11.1 GitHub

[Paste GitHub repository link here]

11.2 Project demo link

[Paste demo video link here]